

1. **Lenguajes de SQL:** Los sublenguajes de SQL · DDL: definir el esquema · DML: leer y modificar filas · DCL: usuarios, roles y privilegios · TCL: transacciones
2. **Restricciones:** PRIMARY KEY, UNIQUE y NOT NULL · FOREIGN KEY y acciones referenciales · CHECK y DEFAULT
3. **Índices:** Qué es un índice · Índices compuestos y de cobertura · Lo que cuestan los índices · Tipos de índice
4. **Agrupamiento:** Índices agrupados y no agrupados · El orden físico en PostgreSQL y Oracle
5. **Particionado:** Particionado horizontal · Poda de particiones y mantenimiento · Particionado vertical y sharding
6. **Eficiencia:** Leer un plan de ejecución · Condiciones que pueden usar un índice · Escribir consultas eficientes

Lenguajes de SQL

Los sublenguajes de SQL

SQL es un solo lenguaje con varios **sublenguajes**, agrupados según lo que hace cada sentencia: definir la estructura, trabajar con los datos, controlar quién puede hacer qué y agrupar los cambios en transacciones.

Los cuatro grupos

Sublenguaje	Sentencias	Sobre qué actúa
DDL · definición de datos	CREATE, ALTER, DROP, TRUNCATE, RENAME	El esquema: tablas, columnas, restricciones, índices, vistas
DML · manipulación de datos	SELECT, INSERT, UPDATE, DELETE, MERGE	Las filas almacenadas en las tablas
DCL · control de datos	GRANT, REVOKE	Usuarios, roles y sus privilegios
TCL · control de transacciones	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	Cuándo los cambios pasan a ser permanentes o se deshacen

Ejemplo. CREATE TABLE student (...) es DDL; INSERT INTO student VALUES (...) es DML; GRANT SELECT ON student TO teacher es DCL; COMMIT es TCL.

Error frecuente. Pensar que TRUNCATE es lo mismo que DELETE sin WHERE. TRUNCATE es DDL: vacía la tabla de golpe, normalmente no admite filtro y en muchos SGBD confirma la transacción de forma implícita.

DDL: definir el esquema

CREATE crea un objeto, **ALTER** lo modifica y **DROP** lo elimina, datos incluidos. Con DDL el modelo lógico se convierte en tablas reales.

- Tipos: INTEGER, NUMERIC(p, s), VARCHAR(n) / TEXT, DATE, TIMESTAMP, BOOLEAN... (cada SGBD tiene su propia lista).
- Cada sentencia DDL actualiza el diccionario de datos, que se puede consultar: aquí, `sqlite_master`.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE course (
  course_id TEXT PRIMARY KEY,
  title     TEXT NOT NULL,
  credits   REAL NOT NULL
);
ALTER TABLE course ADD COLUMN semester INTEGER;
INSERT INTO course VALUES ('C10', 'Databases', 6, 1);
SELECT * FROM course;
SELECT name, sql FROM sqlite_master WHERE type = 'table';
DROP TABLE course;
SELECT name FROM sqlite_master WHERE type = 'table';
```

Error frecuente. Borrar y volver a crear una tabla para cambiar una columna. ALTER TABLE conserva las filas; DROP las pierde.

DML: leer y modificar filas

SELECT lee, **INSERT** añade filas, **UPDATE** las modifica y **DELETE** las elimina. **UPDATE** y **DELETE** afectan a todas las filas que cumplen el **WHERE**.

- Las combinaciones (JOIN) siguen las claves ajenas: `enrolment.course_id = course.course_id`.
- El ejecutor muestra cuántas filas ha cambiado cada INSERT, UPDATE o DELETE.

Ejemplo que puedes ejecutar en la web

```
INSERT INTO enrolment VALUES ('S03', 'C20', 6.0);
UPDATE enrolment SET grade = grade + 0.5 WHERE course_id = 'C10' AND grade < 6;
DELETE FROM enrolment WHERE student_id = 'S02' AND course_id = 'C11';
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course c ON c.course_id = e.course_id
ORDER BY s.name, c.title;
SELECT c.title, COUNT(*) AS students, ROUND(AVG(e.grade), 2) AS average
FROM enrolment e JOIN course c ON c.course_id = e.course_id
GROUP BY c.title
HAVING COUNT(*) >= 2;
```

Ejemplo. La nota media por asignatura es `SELECT course_id, AVG(grade) FROM enrolment GROUP BY course_id`.

Error frecuente. Filtrar un agregado con `WHERE (WHERE AVG(grade) > 7)`. `WHERE` se aplica antes de agrupar; usa `HAVING AVG(grade) > 7`.

DCL: usuarios, roles y privilegios

GRANT concede un privilegio sobre un objeto a un usuario o rol; **REVOKE** lo retira. Da a cada rol solo lo que necesita: el principio de mínimo privilegio.

- `WITH GRANT OPTION` permite a quien lo recibe conceder ese mismo privilegio a otros; úsalo con moderación.
- `REVOKE` retira el privilegio a ese rol; sus miembros lo pierden salvo que otro rol se lo siga concediendo.

PostgreSQL · no se ejecuta aquí

```
CREATE ROLE teacher;
CREATE ROLE reporting;
CREATE USER ana PASSWORD 'change-me' IN ROLE teacher;

GRANT SELECT, INSERT, UPDATE ON enrolment TO teacher;
GRANT SELECT ON student, course TO teacher;

CREATE VIEW course_average AS
  SELECT course_id, AVG(grade) AS average FROM enrolment GROUP BY course_id;
GRANT SELECT ON course_average TO reporting;      -- sin acceso a las notas individuales

REVOKE UPDATE ON enrolment FROM teacher;
```

Error frecuente. Dejar que la aplicación se conecte como propietaria de la base de datos. Un fallo o una inyección SQL puede entonces borrar tablas; con un rol limitado a DML sobre sus tablas, no puede.

TCL: transacciones

Una **transacción** agrupa sentencias que deben tener éxito o fallar juntas. **COMMIT** las confirma (las hace permanentes), **ROLLBACK** las deshace y un **SAVEPOINT** (punto de guardado) marca un punto al que se puede volver.

- `SAVEPOINT nombre ... ROLLBACK TO nombre` deshace solo el trabajo posterior al punto de guardado.
- Mantén las transacciones cortas: mientras una está abierta, puede retener bloqueos que hacen esperar a las demás.

Ejemplo que puedes ejecutar en la web

```
BEGIN;
DELETE FROM enrolment WHERE student_id = 'S03' AND course_id = 'C11';
```

```

INSERT INTO enrolment VALUES ('S03', 'C20', NULL);
SAVEPOINT before_grades;
UPDATE enrolment SET grade = 10;           -- ¡uy!: todas las filas
ROLLBACK TO before_grades;
COMMIT;
SELECT * FROM enrolment WHERE student_id = 'S03';
BEGIN;
DELETE FROM enrolment;
SELECT COUNT(*) AS rows_inside_transaction FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM enrolment;

```

Ejemplo. El segundo bloque cambia una nota por error y vuelve al punto de guardado: el cambio de asignatura se conserva, la nota errónea no.

Error frecuente. Ejecutar un script largo en modo autocommit. Si falla a mitad, la primera mitad queda aplicada y la base de datos se queda en un estado que nadie había previsto.

Restricciones

PRIMARY KEY, UNIQUE y NOT NULL

La **clave primaria** identifica cada fila: única y nunca NULL. **UNIQUE** prohíbe duplicados en otras claves candidatas y **NOT NULL** hace obligatoria una columna.

- Clave compuesta: PRIMARY KEY (student_id, course_id).
- La última sentencia del ejemplo falla a propósito: lee el mensaje de error.

Ejemplo que puedes ejecutar en la web

```

INSERT INTO student VALUES ('S04', 'Marta Paz', NULL);  -- se admite un segundo correo NULL
SELECT * FROM student;
INSERT INTO student VALUES ('S05', 'Ana Ruiz', 'ana@uni.es');  -- correo duplicado

```

Error frecuente. Usar solo un identificador artificial y olvidar la clave natural. Con id como PK y sin UNIQUE (email), el mismo estudiante se puede insertar dos veces.

FOREIGN KEY y acciones referenciales

Una **clave ajena** solo acepta valores que existen en la clave referenciada. Su **acción referencial** indica qué ocurre con las filas hijas cuando se borra la fila padre o cambia su clave.

- CASCADE encaja con partes que no tienen sentido solas (una matrícula sin su estudiante, una línea de pedido sin su pedido).
- RESTRICT protege los datos de referencia (una asignatura, un departamento).
- SQLite solo comprueba las claves ajenas tras PRAGMA foreign_keys = ON; el ejecutor lo activa.

Ejemplo que puedes ejecutar en la web

```

DELETE FROM student WHERE student_id = 'S01';  -- CASCADE: también se van las matrículas de Ana
SELECT * FROM enrolment ORDER BY student_id;
DELETE FROM course WHERE course_id = 'C10';    -- C10 aún tiene matrículas: rechazado

```

Error frecuente. Usar CASCADE en todas partes. Borrar un departamento podría entonces borrar sin avisar sus asignaturas y todas sus matrículas.

CHECK y DEFAULT

CHECK establece una condición que debe cumplir cada fila, como una nota entre 0 y 10. **DEFAULT** da su valor a una columna cuando el INSERT no indica ninguno.

- Un CHECK se supera cuando la condición es verdadera o desconocida (NULL); añade NOT NULL si el valor es obligatorio.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE loan (
  loan_id  INTEGER PRIMARY KEY,
  book     TEXT NOT NULL,
  loan_date TEXT NOT NULL DEFAULT (date('now')),
  days     INTEGER NOT NULL DEFAULT 15,
  status   TEXT NOT NULL DEFAULT 'open',
  CONSTRAINT days_range CHECK (days BETWEEN 1 AND 60),
  CONSTRAINT status_values CHECK (status IN ('open', 'returned', 'lost'))
);
INSERT INTO loan (book) VALUES ('Don Quixote');
INSERT INTO loan (book, days, status) VALUES ('Ficciones', 30, 'returned');
SELECT * FROM loan;
INSERT INTO loan (book, days) VALUES ('Hopscotch', 90);      -- incumple days_range
```

Error frecuente. Comprobar los rangos solo en la aplicación. Los datos cargados con un script o desde otro programa se saltan entonces la regla.

Índices

Qué es un índice

Un **índice** es una estructura aparte y ordenada (normalmente un **árbol B+**) que asocia los valores de algunas columnas con sus filas. Gracias a él, el SGBD puede **buscar** en lugar de **recorrer** toda la tabla.

- La clave primaria y cada restricción UNIQUE ya tienen un índice.
- Las claves ajenas normalmente **no** reciben uno automáticamente; indéxalas si combinas o filtras por ellas.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM customer WHERE email = 'user43210@mail.com';
CREATE INDEX idx_customer_email ON customer (email);
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM customer WHERE email = 'user43210@mail.com';
```

Error frecuente. Pensar que un índice cambia el resultado. Solo cambia la rapidez con que el SGBD encuentra las filas; la consulta devuelve exactamente los mismos datos.

Índices compuestos y de cobertura

Un índice **compuesto** ordena por varias columnas, en orden. Ayuda a las consultas que filtran por sus columnas **más a la izquierda**. Un índice **de cobertura** contiene todas las columnas que necesita una consulta, de modo que **no hace falta leer la tabla**.

- Regla del prefijo izquierdo: (a, b, c) sirve para filtros sobre a, a, b y a, b, c.
- Un índice compuesto puede sustituir a varios de una sola columna.

Ejemplo que puedes ejecutar en la web

```
CREATE INDEX idx_city_date ON customer (city, signup_date);
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE city = 'Madrid' AND signup_date >= '2023-01-01';
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE signup_date >= '2023-01-01';
EXPLAIN QUERY PLAN SELECT city, signup_date FROM customer WHERE city = 'Sevilla';
SELECT COUNT(*) AS madrid_since_2023 FROM customer WHERE city = 'Madrid' AND signup_date >= '2023-01-01';
```

Error frecuente. Crear (signup_date, city) para consultas que siempre fijan la ciudad y dan un rango de fechas. Con el rango primero, la segunda columna no puede acotar la búsqueda.

Lo que cuestan los índices

Los índices **aceleran las lecturas pero ralentizan las escrituras**: cada INSERT, UPDATE o DELETE debe actualizar

también **cada índice** de la tabla. Además, ocupan disco y memoria.

Cuándo indexar una columna

Normalmente sí	Normalmente no
Claves ajenas usadas en combinaciones (JOIN)	Columnas que nunca se usan en WHERE, JOIN u ORDER BY
Columnas filtradas con = o rangos que devuelven pocas filas	Columnas de baja selectividad (sexo, sí/no)
Columnas usadas para ordenar resultados grandes (ORDER BY ... LIMIT)	Tablas pequeñas que caben en pocos bloques
Claves de búsqueda de consultas frecuentes	Tablas con muchas escrituras y pocas lecturas (registros, logs)

Ejemplo que puedes ejecutar en la web

```
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 20000)
INSERT INTO plain_log SELECT i, i % 997, 'action' || (i % 13), datetime('2024-01-01', '+' || i || ' minutes') FROM
n;
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 20000)
INSERT INTO indexed_log SELECT i, i % 997, 'action' || (i % 13), datetime('2024-01-01', '+' || i || ' minutes')
FROM n;
SELECT name AS indexes_on_indexed_log FROM sqlite_master WHERE type = 'index' AND tbl_name = 'indexed_log';
```

Error frecuente. Indexar todas las columnas «por si acaso». Cada índice ralentiza todas las escrituras y el optimizador solo usa los útiles.

Tipos de índice

El **árbol B** es el tipo por defecto y sirve para igualdad, rangos y ordenación. Otros tipos resuelven problemas concretos: **hash** solo para igualdad, **bitmap** para pocos valores distintos, **GIN** para valores dentro de un documento o un array y **BRIN** para tablas enormes ordenadas de forma natural.

Principales tipos de índice

Tipo	Adecuado para	No sirve para
Árbol B / árbol B+	=, <, >, BETWEEN, ORDER BY, LIKE con prefijo 'abc%'	Buscar dentro de textos o arrays
Hash	Solo igualdad (=)	Rangos y ordenación
Bitmap	Columnas con pocos valores distintos en tablas casi solo de lectura (almacenes de datos)	Tablas con muchas escrituras concurrentes
GIN / invertido	Búsqueda de texto completo, claves JSON, elementos de arrays	Columnas escalares simples
BRIN	Tablas muy grandes ordenadas por inserción (fechas de un registro)	Datos en orden aleatorio

PostgreSQL · no se ejecuta aquí

```
CREATE INDEX orders_customer_idx ON orders (customer_id);           -- árbol B (por defecto)
CREATE INDEX session_token_idx  ON session USING hash (token);     -- solo igualdad
CREATE INDEX product_tags_idx   ON product USING gin (tags);       -- tags es text[]
CREATE INDEX log_at_brin        ON access_log USING brin (at);      -- enorme, solo se añade al final
CREATE INDEX open_orders_idx    ON orders (order_date) WHERE status = 'open'; -- parcial
CREATE INDEX customer_email_ci  ON customer (lower(email));        -- sobre expresión
```

Error frecuente. Elegir un índice hash para una columna que se usa en rangos u ORDER BY. No puede resolver ninguno de los dos; solo un árbol B puede.

Agrupamiento

Índices agrupados y no agrupados

En un índice **agrupado** (clustered), las propias filas de la tabla se guardan en el orden de la clave, dentro del índice. Una tabla solo puede tener **uno**. Todos los demás índices son **no agrupados**: guardan la clave más un puntero a la fila.

- Elige una clave de agrupamiento corta, que no cambie y que coincida con las consultas por rango frecuentes.

- Las claves aleatorias (como los UUID aleatorios) dispersan las inserciones por todo el índice agrupado; las claves crecientes se añaden al final.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE enrolment_by_course (
  course_id TEXT,
  student_id TEXT,
  grade REAL,
  PRIMARY KEY (course_id, student_id)
) WITHOUT ROWID;
INSERT INTO enrolment_by_course VALUES
  ('C20', 'S01', 7.0), ('C10', 'S03', 5.0), ('C10', 'S01', 8.5), ('C11', 'S02', 9.0), ('C10', 'S02', 6.5);
SELECT * FROM enrolment_by_course;           -- se guarda, y se devuelve, en el orden de la clave
EXPLAIN QUERY PLAN SELECT * FROM enrolment_by_course WHERE course_id = 'C10';
```

Error frecuente. Confundir un **índice agrupado** (clustered index) con un **clúster de bases de datos**. Lo segundo es un grupo de servidores que trabajan juntos (replicación, alta disponibilidad), un significado de la palabra que no tiene nada que ver.

El orden físico en PostgreSQL y Oracle

PostgreSQL guarda las tablas como montículos (heaps) sin orden. Su orden CLUSTER reescribe una vez la tabla en el orden de un índice, pero las filas nuevas no mantienen ese orden. Oracle ofrece tablas organizadas por índice y clústeres de tablas.

PostgreSQL · no se ejecuta aquí

```
CREATE INDEX orders_date_idx ON orders (order_date);
CLUSTER orders USING orders_date_idx;      -- reescribe la tabla en orden de fecha (la bloquea)
ANALYZE orders;                           -- actualiza las estadísticas tras la reescritura

-- Hasta qué punto el orden físico sigue a una columna (1 = perfectamente ordenada):
SELECT attname, correlation FROM pg_stats
WHERE tablename = 'orders' AND attname = 'order_date';
```

Error frecuente. Esperar que CLUSTER mantenga la tabla ordenada. Es una reescritura puntual; solo un verdadero índice agrupado conserva el orden a medida que llegan filas.

Particionado

Particionado horizontal

El **particionado** divide una tabla grande en trozos más pequeños, las **particiones**, según una **clave de particionado**. Las consultas siguen usando el nombre de la tabla única; el SGBD envía cada fila a su partición.

Cómo elegir el método

Método	Clave típica	Úsalo cuando
Por rango	Fecha, rangos de identificadores	Las consultas y la retención de datos van por tiempo
Por lista	País, región, estado	Unos pocos valores conocidos separan los datos
Por hash	Identificador de cliente	Necesitas tamaños parecidos y no hay un rango natural

PostgreSQL · no se ejecuta aquí

```
CREATE TABLE orders (
  order_id   bigint,
  customer_id bigint NOT NULL,
  order_date date  NOT NULL,
  total      numeric(10, 2) NOT NULL,
  PRIMARY KEY (order_id, order_date)      -- la clave de particionado debe formar parte de ella
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2025_01 PARTITION OF orders
FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

```
CREATE TABLE orders_2025_02 PARTITION OF orders
  FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;  -- filas fuera de todos los rangos

CREATE INDEX ON orders (customer_id);  -- se crea en cada partición
```

Error frecuente. Particionar una tabla pequeña. Por debajo de muchos millones de filas, un buen índice suele ser más sencillo e igual de rápido.

Poda de particiones y mantenimiento

Cuando el **WHERE** fija la clave de particionado, el optimizador lee solo las particiones que coinciden: la **poda de particiones** (partition pruning). Los datos antiguos se pueden eliminar entonces borrando una partición entera en lugar de millones de filas.

- Elige la clave de particionado a partir de los filtros más frecuentes.
- Las restricciones de unicidad y la clave primaria deben incluir la clave de particionado.

PostgreSQL · no se ejecuta aquí

```
EXPLAIN SELECT SUM(total) FROM orders
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01';
-- Aggregate
--   -> Seq Scan on orders_2025_02 orders    <- solo se lee una partición

ALTER TABLE orders DETACH PARTITION orders_2025_01;  -- se conserva como tabla normal...
DROP TABLE orders_2025_01;  -- ...o se elimina un mes entero de una vez
```

Error frecuente. Particionar por una columna por la que las consultas casi nunca filtran. Sin poda, cada consulta visita todas las particiones.

Particionado vertical y sharding

El particionado **vertical** divide las **columnas** de una tabla en dos tablas con la misma clave. El **sharding** (fragmentación horizontal entre servidores) reparte las **filas** entre distintos **servidores**, cada uno con su propia base de datos.

Tres formas de dividir

Técnica	Divide	Dónde están los trozos	Coste principal
Particionado horizontal	Filas	La misma base de datos	Las consultas sin la clave visitan todas las particiones
Particionado vertical	Columnas	La misma base de datos	Una combinación para leer la fila completa
Sharding	Filas	Servidores distintos	Combinaciones y transacciones entre shards

Error frecuente. Recurrir al sharding antes de necesitarlo. Los índices, el particionado, las réplicas de lectura y un servidor más potente resuelven la mayoría de los problemas con mucha menos complejidad.

Eficiencia

Leer un plan de ejecución

SQL dice **qué** quieres; el **optimizador** decide **cómo**: qué índice, qué orden de combinación, qué algoritmo. **EXPLAIN** muestra ese plan, y en el plan es donde se entienden las consultas lentas.

- SCAN = leer todas las filas; SEARCH ... USING INDEX = bajar por un índice.
- USE TEMP B-TREE FOR ORDER BY = una ordenación adicional; un índice en ese orden la evita.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN QUERY PLAN
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

```
CREATE INDEX idx_orders_customer ON orders (customer_id);
ANALYZE;
EXPLAIN QUERY PLAN
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

Error frecuente. Optimizar por intuición. Lee primero el plan: la parte lenta a menudo no está donde esperas.

Condiciones que pueden usar un índice

Un índice sobre una columna solo ayuda cuando la condición compara la columna tal cual. Envolverla en una función o un cálculo la oculta al índice: la consulta recorre la tabla.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM customer WHERE substr(signup_date, 1, 4) = '2023';
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM customer WHERE signup_date >= '2023-01-01' AND signup_date < '2024-01-01';
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE lower(email) = 'user77@mail.com';
CREATE INDEX idx_email_lower ON customer (lower(email));
EXPLAIN QUERY PLAN SELECT * FROM customer WHERE lower(email) = 'user77@mail.com';
```

Error frecuente. Añadir un índice y dar por hecho que la consulta lo usará. Comprueba el plan; basta una función sobre la columna para que el índice no sirva.

Escribir consultas eficientes

Pide a la base de datos exactamente lo que necesitas, en una sola sentencia orientada a conjuntos, y deja que haga el trabajo cerca de los datos: filtra, combina y agrega en SQL, no fila a fila en la aplicación.

- Selecciona solo las columnas que usas; `SELECT *` lee y envía más datos e impide que los índices de cobertura ayuden.
- Usa `EXISTS` para preguntar «¿hay alguno?» en lugar de `COUNT(*) > 0`: puede detenerse en la primera coincidencia.
- Página con `ORDER BY ... LIMIT`, idealmente sobre una columna indexada.
- Combina y filtra por claves indexadas, y mantén las transacciones cortas.

Ejemplo que puedes ejecutar en la web

```
-- «¿Ha hecho algún pedido el cliente 42?»: EXISTS puede detenerse en el primer pedido
SELECT EXISTS (SELECT 1 FROM orders WHERE customer_id = 42) AS has_orders;
-- Una sentencia orientada a conjuntos en lugar de un bucle sobre cada pedido
UPDATE orders SET total = round(total * 0.9, 2) WHERE order_date < '2023-02-01';
-- Solo las columnas y filas necesarias, agregadas en la base de datos
SELECT c.city, COUNT(*) AS orders, ROUND(SUM(o.total), 2) AS spent
FROM orders o JOIN customer c ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.city
ORDER BY spent DESC
LIMIT 5;
```

Ejemplo. La última consulta del ejemplo responde a «las 5 ciudades con más gasto en 2024» con una sola sentencia, en lugar de cargar todos los pedidos en el programa.

Error frecuente. Traer una tabla entera para filtrarla en la aplicación. La base de datos puede usar índices y enviar solo las filas que coinciden; el programa no puede hacer ninguna de las dos cosas.